

Motion planning for obstacle navigation and detection for an e-puck Robot

Yichen Benjamin Zhou¹ and Yuxin Liu²

Abstract—Mobile robots require quick and reliable responses to navigate and track objects or obstacles. This report presents reactive control strategies for an e-puck mobile robot in a confined arena and object tracking in an open space. Finite state machines are implemented using infrared proximity sensing for directional detection and time-of-flight distance sensing for range estimation. Proximity thresholds, pivot-turns, and distance-based approaches resulted in consistent obstacle navigation and object tracking. Experimental results show the robot displayed smooth and stable motion, maintained safe separation distances, and avoided collisions under all the tested conditions.

I. STRATEGIES

A. Obstacle Avoidance

The obstacle-avoidance strategy is based on a reactive approach where the robot continuously drives forward and responds immediately to proximity sensor detections. The core idea being the use of directional infrared (IR) sensing to classify environmental situations into two repeatable categories that map directly to the motor rotational velocities. Two categories of obstruction are considered: a frontal obstruction that prevents forward motion, and a lateral obstruction that indicates the robot is drifting too close to a wall. A state transition diagram is shown in Fig. 1.

When a frontal obstruction is detected, the robot pivots about its own axis in the direction that provides the greatest clearance. This avoids the complexity of curved paths and ensures predictable, reversible behaviour in confined spaces. When a lateral wall is detected, the robot applies a differential adjustment to its wheel speeds, steering smoothly away from the wall while maintaining forward motion. These reactive rules enable the robot to navigate the arena without colliding, or requiring any internal mapping, memory, or long term planning. The strategy relies entirely on threshold based decisions derived from real-time sensor measurements, making it simple, robust, and suited for application in controlled environments.

B. Object Tracking

The object-tracking strategy extends the reactive approach by combining time-of-flight (ToF) distance sensing with the IR proximity sensors to determine the object's bearing relative to the robot. The task is decomposed into five behaviours: searching for the object, aligning to face it,

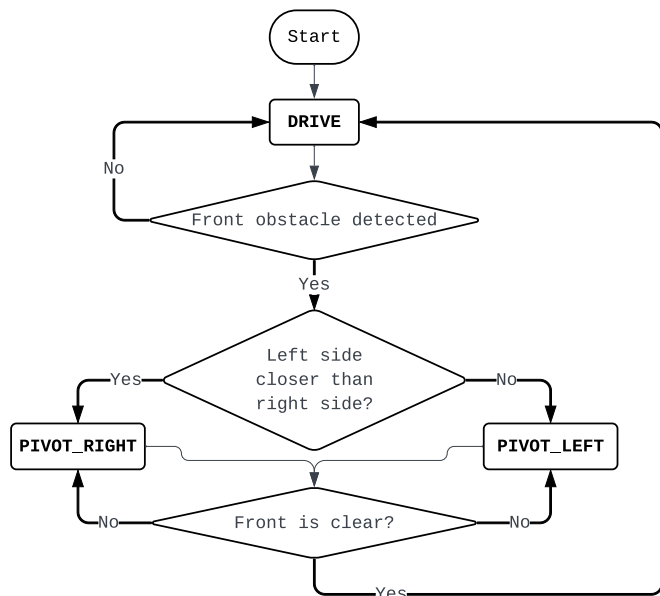


Fig. 1. Obstacle Avoidance state transitions

approaching it, stopping when at a safe distance, and backing off when the object is too close. A state transition diagram is shown in Fig. 2.

When the object is not detected, the robot spins about its own axis until the ToF sensor identifies a valid target. Once in range, the IR proximity sensors are used to determine whether the object lies directly ahead or off-centre. If it is off-centre, the robot pivots in place to reorient toward the object. If the object is centered, the robot approaches forward, stopping when in a predefined safe zone, and re-approaching or backing off when depending how which side of the safe zone the object is moved to. This enables the robot to maintain a stable final position without overshooting. Each behaviour directly links sensor readings to motor velocities, making the strategy fully reactive.

II. IMPLEMENTATION

Both tasks were implemented using finite state machines (FSM), with each behavioural mode represented as a distinct state and all transitions triggered by real-time sensor evaluation as seen in Fig. 1 and Fig. 2. This structure allows the control logic to remain modular, with each state encapsulating a single, defined behaviour which allows for ease of interpretation and debugging. The full source code, programmed in C, is provided in the Appendix. The following subsections describe how each strategy is implemented

*This work was not supported by any organization

¹Zhou is with the Department of Automatic Control and Systems Engineering, The University of Sheffield, UK yblzhou1@sheffield.ac.uk

²Liu is with the Department of Automatic Control and Systems Engineering, The University of Sheffield, UK yliu684@sheffield.ac.uk

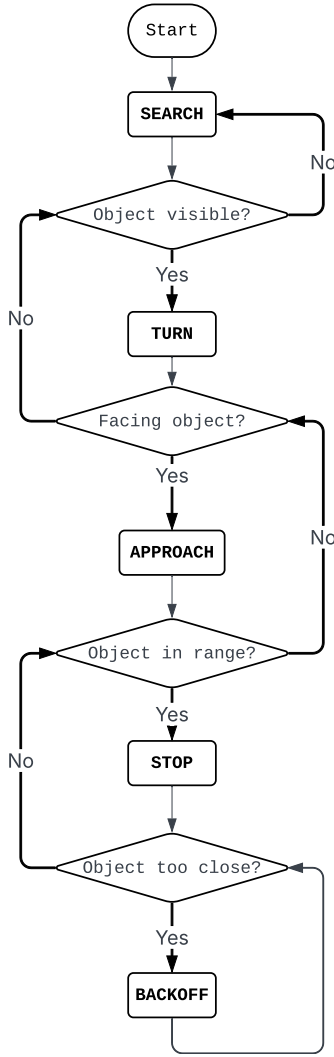


Fig. 2. Object Tracking state transitions

in software.

A. Obstacle Avoidance

The obstacle avoidance behaviour is implemented through the function `run_obstacle_avoidance()`, which repeatedly polls the infrared proximity sensors and updates the state of the FSM. The `DRIVE`, `PIVOT_LEFT`, and `PIVOT_RIGHT` states correspond to specific motor commands, defined by fixed-speed forward motion or pivot-on-spot rotations.

The `DRIVE` state incorporates proportional side-correction computed from the lateral proximity sensors as seen in lines 76-81 of the code. It adjusts the left and right motor speeds in opposite directions, steering the robot away from a lateral wall while preserving forward motion. When the front-left or front-right sensors exceed their thresholds, the system transitions to a pivot state, determined by comparing the relative proximity values on each side. The two pivot states maintain the turning motion until the front sensor outputs fall below the release threshold, at which point the robot returns to `DRIVE`. This structure guarantees predictable transitions

and avoids oscillatory behaviour through the inclusion of a small hold counter.

B. Object Tracking

Object tracking is implemented in `run_object_chasing()`, where the ToF sensor is used to detect and confirm the target's presence and distance. The proximity sensors subsequently determine its angular position once the object is within their range.

In the `SEARCH` state, the robot performs a scan of its environment by rotating about its own axis by commanding of opposing motor speeds. Once the ToF sensor confirms an object in range, the FSM transitions to `TURN`. The `TURN` state realigns the robot, if needed, by pivoting in the direction of the strongest proximity reading. In the case that the target lies directly ahead, the FSM moves into `APPROACH`, where forward motion is applied. A graded speed command is used, with the approach velocity decreasing as the robot nears the desired distance range, which improves stability.

The `STOP` state holds the robot stationary while continuously checking for object displacement. If the object moves laterally, a small tracking rotation is applied. If it moves too close or disappears, it transitions to `BACKOFF` or `SEARCH` respectively. `BACKOFF` slowly drives the robot in reverse to restore safe separation before returning to `STOP`. This implementation directly maps the sensing conditions to motor outputs, leading to smooth and predictable target engagement.

III. RESULTS & DISCUSSIONS

Both strategies were evaluated in the laboratory environments used for the final demonstration.

A. Obstacle Avoidance Results

Across multiple trials, the robot successfully navigated the obstacle arena without contacting any walls while driving at approximately 72% of maximum wheel speed. A consistent limitation was observed when approaching wall corners at specifically shallow angles (approximately 100 relative to the wall normal). In this configuration, both front facing and side facing proximity sensors occasionally failed to register the corner, resulting in collision. This is suggestive of a blind region which may be tunable through further testing and adjustment of proximity threshold values.

B. Object Tracking Results

Although the object-tracking strategy produced stable and predictable behaviour, the smoothness of motion during transitions between states could be improved. Since state transitions occur abruptly, resulting commands cause the robot to move in short, discontinuous bursts. A more refined control scheme could mitigate this by applying velocity commands proportional to the measured rate of change in distance. This would produce a smoother trajectory while preserving the responsiveness of the reactive controller.

IV. APPENDIX

The complete C source code used for the e-puck robot control is provided below.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <math.h>
5
6 #include "ch.h"
7 #include "hal.h"
8 #include "memory_protection.h"
9 #include <main.h>
10
11 #include "leds.h"
12 #include "spi_comm.h"
13 #include "sensors/proximity.h"
14 #include "sensors/VL53L0X/VL53L0X.h"
15 #include "motors.h"
16 #include "sensors/battery_level.h"
17 #include "audio/audio_thread.h"
18 #include "audio/play_melody.h"
19 #include "audio/play_sound_file.h"
20 #include "selector.h"
21
22 messagebus_t bus;
23 MUTEX_DECL(bus_lock);
24 CONDVAR_DECL(bus_condvar);
25
26 // --- TASK 1 (AVOIDANCE) --- //
27 typedef enum { DRIVE = 0, PIVOT_LEFT,
    PIVOT_RIGHT } AvoidState;
28
29 const int T1_THR_FRONT = 700;
30 const int T1_REL_FRONT = 500;
31 const int T1_THR_SIDE = 600;
32 const int T1_DRIVE_SPEED = 650;
33 const int T1_PIVOT_SPEED = 650;
34 const int T1_SIDE_GAIN = 1;
35 const int T1_HOLD_CYCLES = 3;
36 const int LOOP_DT_MS = 80;
37
38 static void set_rgb_task1(AvoidState s)
39 {
40     clear_leds();
41     switch(s){
42         case DRIVE: set_rgb_led(0,0,1,0); break;
43             // Green
44         case PIVOT_LEFT: set_rgb_led(0,0,0,1);
45             break; // Blue
46         case PIVOT_RIGHT: set_rgb_led(0,1,0,0);
47             break; // Red
48         default: break;
49     }
50 }
51
52 static void update_ring_leds_task1(int F,int L,
    int R)
53 {
54     set_led(LED1, (F > T1_THR_FRONT) ? 1 : 0);
55     set_led(LED3, (R > T1_THR_SIDE) ? 1 : 0);
56     set_led(LED7, (L > T1_THR_SIDE) ? 1 : 0);
57     set_led(LED5, 0);
58 }
59
60 void run_obstacle_avoidance(void)
61 {
62     AvoidState state = DRIVE;
63     int hold_counter = 0;
64     set_rgb_task1(state);
65
66     while (get_selector() == 0) {
67         int p0=get_prox(0),p1=get_prox(1),p2=
68             get_prox(2);
69         int p5=get_prox(5),p6=get_prox(6),p7=
69             get_prox(7);
70
71         int F=(p0+p7)/2, L=(p6+p5+p7)/3, R=(p0+p1+
72             p2)/3;
73
74         update_ring_leds_task1(F,L,R);
75
76         switch(state){
77             case DRIVE:
78                 if(F>=T1_THR_FRONT){
79                     state=(L>=R)?PIVOT_RIGHT:PIVOT_LEFT;
80                     hold_counter=T1_HOLD_CYCLES;
81                     set_rgb_task1(state);
82                     left_motor_set_speed((state==
83                         PIVOT_LEFT)?-T1_PIVOT_SPEED:
84                         T1_PIVOT_SPEED);
85                     right_motor_set_speed((state==
86                         PIVOT_LEFT)? T1_PIVOT_SPEED:-
87                         T1_PIVOT_SPEED);
88                 }
89                 else{
90                     int ls=T1_DRIVE_SPEED,rs=
91                         T1_DRIVE_SPEED;
92                     if(L>T1_THR_SIDE && L>R+50){ ls+=
93                         T1_SIDE_GAIN*(L-T1_THR_SIDE); rs
94                         -=T1_SIDE_GAIN*(L-T1_THR_SIDE);
95                     }
96                     else if(R>T1_THR_SIDE && R>L+50){ ls
97                         -=T1_SIDE_GAIN*(R-T1_THR_SIDE);
98                         rs+=T1_SIDE_GAIN*(R-T1_THR_SIDE)
99                         ; }
100                     if(ls>1000)ls=1000;if(rs>1000)rs
101                         =1000;if(ls<0)ls=0;if(rs<0)rs=0;
102                     left_motor_set_speed(ls);
103                     right_motor_set_speed(rs);
104                 }
105                 break;
106             case PIVOT_LEFT:
107             case PIVOT_RIGHT:
108                 left_motor_set_speed((state==PIVOT_LEFT
109                     )?-T1_PIVOT_SPEED: T1_PIVOT_SPEED);
110                 right_motor_set_speed((state==
111                     PIVOT_LEFT)? T1_PIVOT_SPEED:-
112                     T1_PIVOT_SPEED);
113                 if(F<T1_REL_FRONT){
114                     if(hold_counter>0) hold_counter--;
115                     else{ state=DRIVE; set_rgb_task1(
116                         state); }
117                 }
118                 else hold_counter=T1_HOLD_CYCLES;
119                 break;
120         }
121         chThdSleepMilliseconds(LOOP_DT_MS);
122     }
123     clear_leds();
124     left_motor_set_speed(0);
125     right_motor_set_speed(0);
126 }
127
128 // --- TASK 2 (CHASE) --- //
129 typedef enum { SEARCH=0, TURN, APPROACH, STOP,
    BACKOFF } ChaseState;
130
131 const uint16_t SEARCH_RANGE_THR = 50;
132 const uint16_t DESIRED_MIN_MM = 30;
133 const uint16_t DESIRED_MAX_MM = 45;
134 const uint16_t BACKOFF_DIST_MM = 25;
135 const uint16_t SAFE_DIST_MM = 35;
136
137 const uint16_t VIS_MAX_MM = 350;
138 const int DETECT_THR = 300;
139 const int LOST_THR = 200;
140
141 const int SEARCH_SPEED = 250;
142 const int TURN_SPEED = 450;
143 const int APPROACH_FAST = 700;
144 const int APPROACH_SLOW = 250;
145 const int BACKOFF_SPEED = -300;
146 const int TRACK_SPEED = 250;

```

```

120
121 const int FRONT_SOFT_THR = 700;
122 const int FRONT_CRIT_THR = 900;
123 const int CENTER_MARGIN = 120;
124
125 static inline int front_read(int p0,int p7,int
    p1,int p6){return (45*p7+45*p0+5*p6+5*p1)
    /100;}
126 static inline int left_read (int p6,int p5,int
    p7){return (60*p6+30*p5+10*p7)/100;}
127 static inline int right_read(int p1,int p2,int
    p0){return (60*p1+30*p2+10*p0)/100;}
128 static inline int back_read (int p3,int p4){
    return (50*p3+50*p4)/100;}
129
130 static void set_led_task2(ChaseState s)
131 {
132     clear_leds();
133     switch(s){
134         case SEARCH: set_rgb_led(0,0,0,1); break;
135             // Blue
136         case TURN: set_rgb_led(0,0,1,1); break; //
137             Cyan
138         case APPROACH: set_rgb_led(0,0,1,0); break;
139             // Green
140         case STOP: set_rgb_led(0,1,1,0); break; //
141             Yellow
142         case BACKOFF: set_rgb_led(0,1,0,0); break;
143             // Red
144         default: break;
145     }
146 }
147
148 void run_object_chasing(void)
149 {
150     ChaseState state = SEARCH;
151     set_led_task2(state);
152
153     while (get_selector() == 1) {
154         uint16_t dist_mm = VL53L0X_get_dist_mm();
155         if(dist_mm==0) dist_mm=9999;
156
157         int p[8]; for(int i=0;i<8;i++) p[i]=
            get_prox(i);
158         int F=front_read(p[0],p[7],p[1],p[6]);
159         int L=left_read(p[6],p[5],p[7]);
160         int R=right_read(p[1],p[2],p[0]);
161         int B=back_read(p[3],p[4]); (void)B;
162         int max_idx=0; for(int i=1;i<8;i++) if(p[i]
            >p[max_idx]) max_idx=i;
163         int max_val=p[max_idx];
164
165         int target_visible = (dist_mm < VIS_MAX_MM
            );
166         int prox_present = (max_val > DETECT_THR);
167         int front_right = (p[0] + p[1] + p[2]) /
            3;
168         int front_left = (p[5] + p[6] + p[7]) / 3;
169         int front_any = (front_right > front_left
            ? front_right : front_left);
170
171         switch(state){
172             case SEARCH:
173                 left_motor_set_speed( SEARCH_SPEED);
174                 right_motor_set_speed(-SEARCH_SPEED);
175                 if(target_visible || prox_present){
176                     state = TURN; set_led_task2(state);
177                 }
178                 break;
179             case TURN:
180                 if(!target_visible && max_val <
                    LOST_THR){
181                     state = SEARCH; set_led_task2(state)
                    ; break;
182                 }
183             }
184     }
185 }

```

```

178
179     if(max_idx==3||max_idx==4){
180         left_motor_set_speed( TURN_SPEED);
181         right_motor_set_speed(-TURN_SPEED);
182     }
183     else if(max_idx==5||max_idx==6||max_idx
        ==7){ left_motor_set_speed(-
        TURN_SPEED); right_motor_set_speed(
        TURN_SPEED); }
184     else if(max_idx==0||max_idx==1||max_idx
        ==2){ left_motor_set_speed(
        TURN_SPEED); right_motor_set_speed
        (-TURN_SPEED); }
185
186     if((max_idx==0||max_idx==7) &&
        target_visible){
187         left_motor_set_speed(0);
188         right_motor_set_speed(0);
189         state = APPROACH; set_led_task2(
            state);
190     }
191     break;
192
193 case APPROACH:
194     if(!target_visible && max_val <
        LOST_THR){
195         left_motor_set_speed(0);
196         right_motor_set_speed(0);
197         state = SEARCH; set_led_task2(state)
            ; break;
198     }
199     if(front_any >= FRONT_CRIT_THR){ state
        = BACKOFF; set_led_task2(state);
200     }
201     if(front_any >= FRONT_SOFT_THR){
202         left_motor_set_speed(0);
203         right_motor_set_speed(0); state =
            TURN; set_led_task2(state); break;
204     }
205     if(dist_mm <= DESIRED_MIN_MM){
206         left_motor_set_speed(0);
207         right_motor_set_speed(0); state =
            STOP; set_led_task2(state); break;
208     }
209
210     // new pivot-then-go logic
211     {
212         int diff = (R - L);
213         if(abs(diff) > CENTER_MARGIN){
214             if(diff > 0){
215                 left_motor_set_speed(
                    TURN_SPEED);
216                 right_motor_set_speed(-
                    TURN_SPEED); }
217             else{ left_motor_set_speed(-
                    TURN_SPEED);
218                 right_motor_set_speed(
                    TURN_SPEED); }
219             break;
220         }
221     }
222
223     if(dist_mm > DESIRED_MAX_MM &&
        target_visible){
224         int denom = (int)SEARCH_RANGE_THR -
            (int)DESIRED_MAX_MM;
225         if(denom < 1) denom = 1;
226         int sp = APPROACH_SLOW +
            ((int)dist_mm - (int)
            DESIRED_MAX_MM) * (
            APPROACH_FAST -
            APPROACH_SLOW) / denom;
227         if(sp > APPROACH_FAST) sp =
            APPROACH_FAST;
228         if(sp < APPROACH_SLOW) sp =

```

```

215     APPROACH_SLOW;
216     left_motor_set_speed(sp);
217     right_motor_set_speed(sp);
218 } else {
219     left_motor_set_speed(0);
220     right_motor_set_speed(0);
221     state = STOP; set_led_task2(state);
222 }
223 break;
224 case STOP:
225     left_motor_set_speed(0);
226     right_motor_set_speed(0);
227 if(!target_visible && max_val <
    LOST_THR){ state = SEARCH;
    set_led_task2(state); break; }
228 if(target_visible && dist_mm <
    BACKOFF_DIST_MM){ state = BACKOFF;
    set_led_task2(state); break; }
229 if(target_visible && dist_mm >
    DESIRED_MAX_MM){ state = APPROACH;
    set_led_task2(state); break; }
230
231 {
232     int TRACK_DIFF_THR = 120;
233     int diffLR = L - R;
234     if(abs(diffLR) > TRACK_DIFF_THR &&
    max_val > DETECT_THR){
235         if(diffLR > 0){
236             left_motor_set_speed(-
    TRACK_SPEED);
237             right_motor_set_speed(
    TRACK_SPEED); }
238         else{ left_motor_set_speed(
    TRACK_SPEED);
239             right_motor_set_speed(-
    TRACK_SPEED); }
240     }else{ left_motor_set_speed(0);
    right_motor_set_speed(0); }
241 }
242 break;
243 case BACKOFF:
244     left_motor_set_speed(BACKOFF_SPEED);
245     right_motor_set_speed(BACKOFF_SPEED
    );
246 if(dist_mm > SAFE_DIST_MM){
247     left_motor_set_speed(0);
248     right_motor_set_speed(0); state =
    STOP; set_led_task2(state); }
249 if(!target_visible && max_val <
    LOST_THR){ left_motor_set_speed(0);
    right_motor_set_speed(0); state =
    SEARCH; set_led_task2(state); }
250 break;
251 }
252 chThdSleepMilliseconds(LOOP_DT_MS);
253 }
254 clear_leds();
255 left_motor_set_speed(0);
256 right_motor_set_speed(0);
257 }
258 // --- MAIN LOOP --- //
259 int main(void)
260 {
261     halInit(); chSysInit(); mpu_init();
262     messagebus_init(&bus,&bus_lock,&bus_condvar);
263
264     clear_leds();
265     spi_comm_start();
266     proximity_start(0); calibrate_ir();
267     VL53L0X_start();
268     motors_init();
269     battery_level_start(); get_battery_percentage

```

```

    );
264     dac_start(); playMelodyStart();
265
266     while (1) {
267         int sel = get_selector();
268         if(sel==0) run_obstacle_avoidance();
269         else if(sel==1) run_object_chasing();
270         else {
271             clear_leds();
272             left_motor_set_speed(0);
273             right_motor_set_speed(0);
274         }
275         chThdSleepMilliseconds(100);
276     }
277 }
278
279 #define STACK_CHK_GUARD 0xe2dee396
280 uintptr_t __stack_chk_guard=STACK_CHK_GUARD;
281 void __stack_chk_fail(void){chSysHalt("Stack_
    smashing_detected");}

```

ACKNOWLEDGMENT

(Acknowledgements...)

REFERENCES

- [1] G. O. Young, "Synthetic structure of industrial plastics (Book style with paper title and editor)", in *Plastics*, 2nd ed. vol. 3, J. Peters, Ed. New York: McGraw-Hill, 1964, pp. 15D64.
- [2] W.-K. Chen, *Linear Networks and Systems* (Book style). Belmont, CA: Wadsworth, 1993, pp. 123D135.